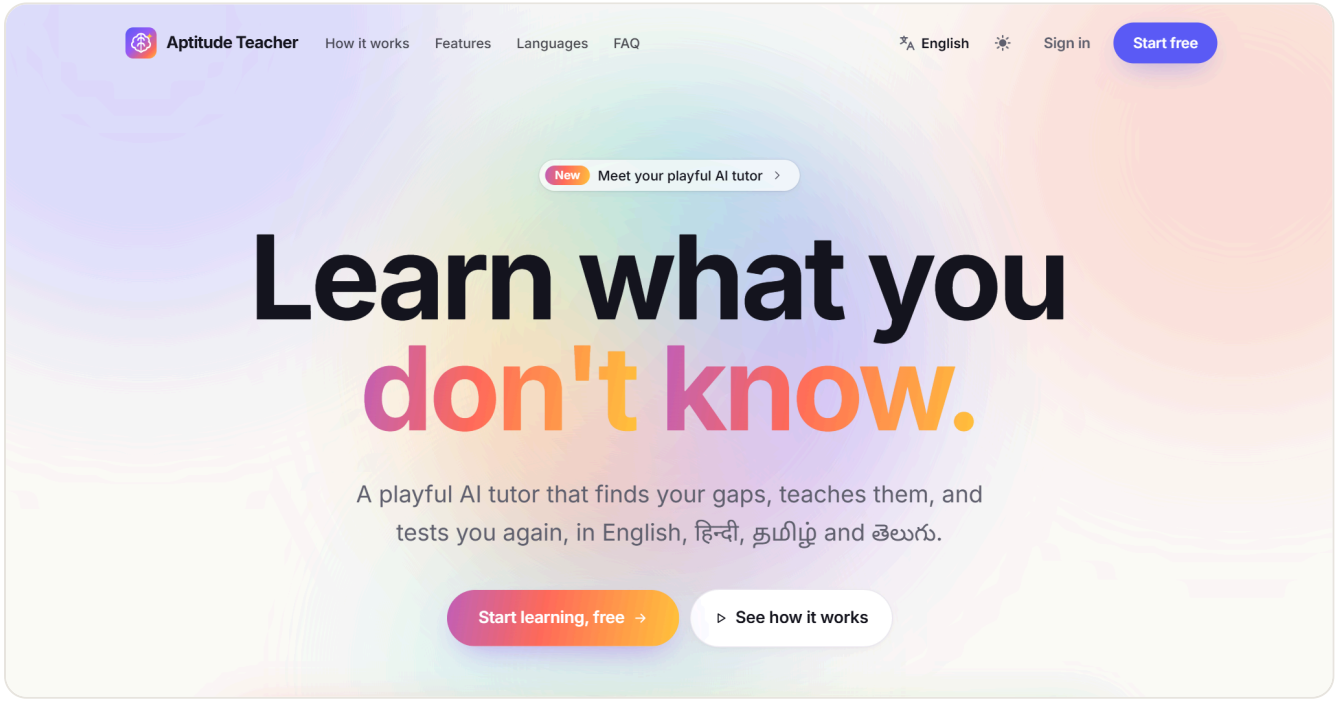
Aptitude Teacher

Project & Deployment Guide

Everything a new person needs to understand, run and put this app online, written for beginners. What it is, how it is built and why, how to install it on a computer, and how to deploy it for free, step by step.

1,212	4	250	₹0
practice questions	languages: English, Hindi, Tamil, Telugu	automatic tests, all passing	to run and host (free tiers)



The public home page of Aptitude Teacher.

Version: October 2026 · Code: private GitHub repository `rhythmoflens007-byte/aptitudeteacher` (branch `main`)

CONTENTS

What is in this guide

01 What Aptitude Teacher is
The problem, the solution, who it is for

02 A tour of the app
Every screen, with pictures

03 The big picture
How the pieces connect

04 Tech stack and why
Each tool and the reason it was chosen

05 How the main parts work
Learning loop, course, game, AI, languages, voice

06 Project map
What every folder does

07 Install on a computer
From a blank PC to a running app

08 Deploy online from scratch
Neon → Render → Vercel, click by click

09 All settings in one table
What each one is and where it comes from

10 After going live
Updates, limits, backups, keys

11 Troubleshooting
"I see X" → "do Y"

12 Security rules
What must never happen

13 Glossary and cheat sheet
Plain meanings of the words used

How to read this

If you only need to put the app online, read chapters 3, 8, 9 and 11. If you need to change the app, read everything. Words you may not know are explained in the glossary (chapter 13).

CHAPTER 1

What Aptitude Teacher is

A friendly, Duolingo-style app that teaches aptitude (maths, reasoning and English) for job and entrance exams, in the student's own language, with a personal AI tutor.

The problem it solves

Students struggle alone

Lakhs of students in India prepare for aptitude rounds (campus placements, bank, SSC, railway exams). Most material is in English, is a wall of questions, and never tells a student *why* they keep getting something wrong.

Practice is boring

Plain question banks feel like homework. Students start, lose motivation in a few days and stop. There is nothing pulling them back every day.

Language barrier

Many students think in Hindi, Tamil or Telugu. An explanation in English they half-understand does not fix the gap.

Tutors are costly

A personal teacher who watches every mistake and explains patiently is not affordable for most students.

What the app does about it

- **Finds the real reason for a mistake.** When a student answers wrongly, the app works out whether it was a careless slip, a misunderstood idea, a half-finished method, or a guess, and responds differently to each.
- **Teaches exactly that gap** with a short lesson, then **tests it again** with a fresh question, and keeps adapting what comes next.
- **Feels like a game:** a step-by-step learning path, hearts, XP points, daily streaks, daily quests, badges, weekly leagues and friends, just like Duolingo.
- **Speaks four languages:** the whole app, the explanations and the tutor work in English, Hindi, Tamil and Telugu.
- **Has a personal tutor** (Asha or Arjun, an animated character) who chats, explains out loud with lip-sync and gestures, listens to voice questions and even reads a photo of a question.
- **Costs nothing to run** at the start: every service used has a free plan.

Who it is for

College students and job seekers preparing for aptitude tests, especially those more comfortable in Hindi, Tamil or Telugu. It works on laptops and phones (it can be "installed" on a phone from the browser, like an app).

The three subjects

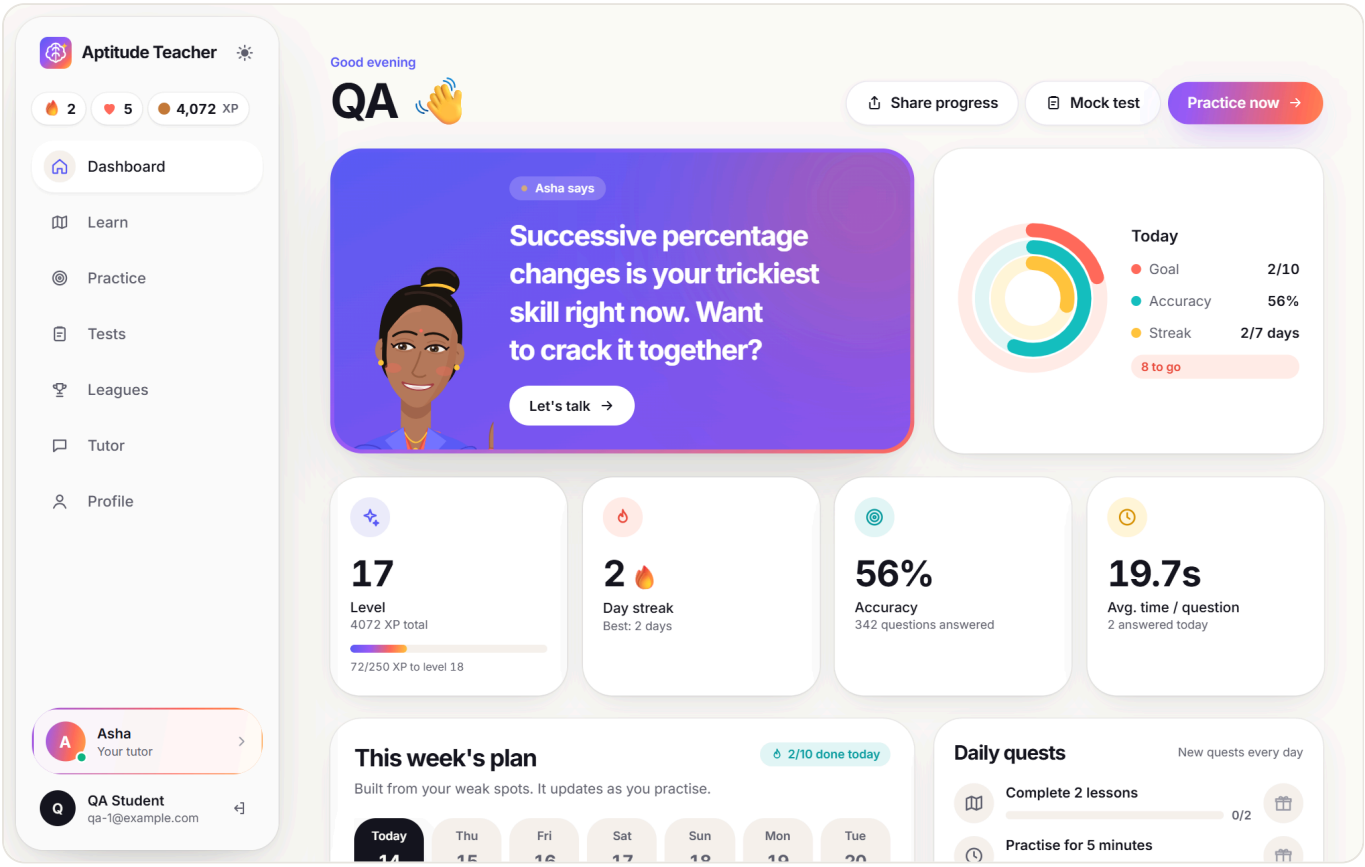
Section	Examples of topics
Quantitative Aptitude	Percentages, profit and loss, time and work, speed and distance, ratios, averages, interest, number system…
Logical Reasoning	Series, coding-decoding, blood relations, directions, syllogisms, statements and assumptions…
Verbal Ability	Synonyms, antonyms, sentence correction, fill in the blanks, reading comprehension…

In total: **3 sections, 28 subtopics, 38 skills, 1,212 questions** (355 written by hand, the rest produced by about 110 "question templates" that create fresh numbers each time), and **12 badges** to earn.

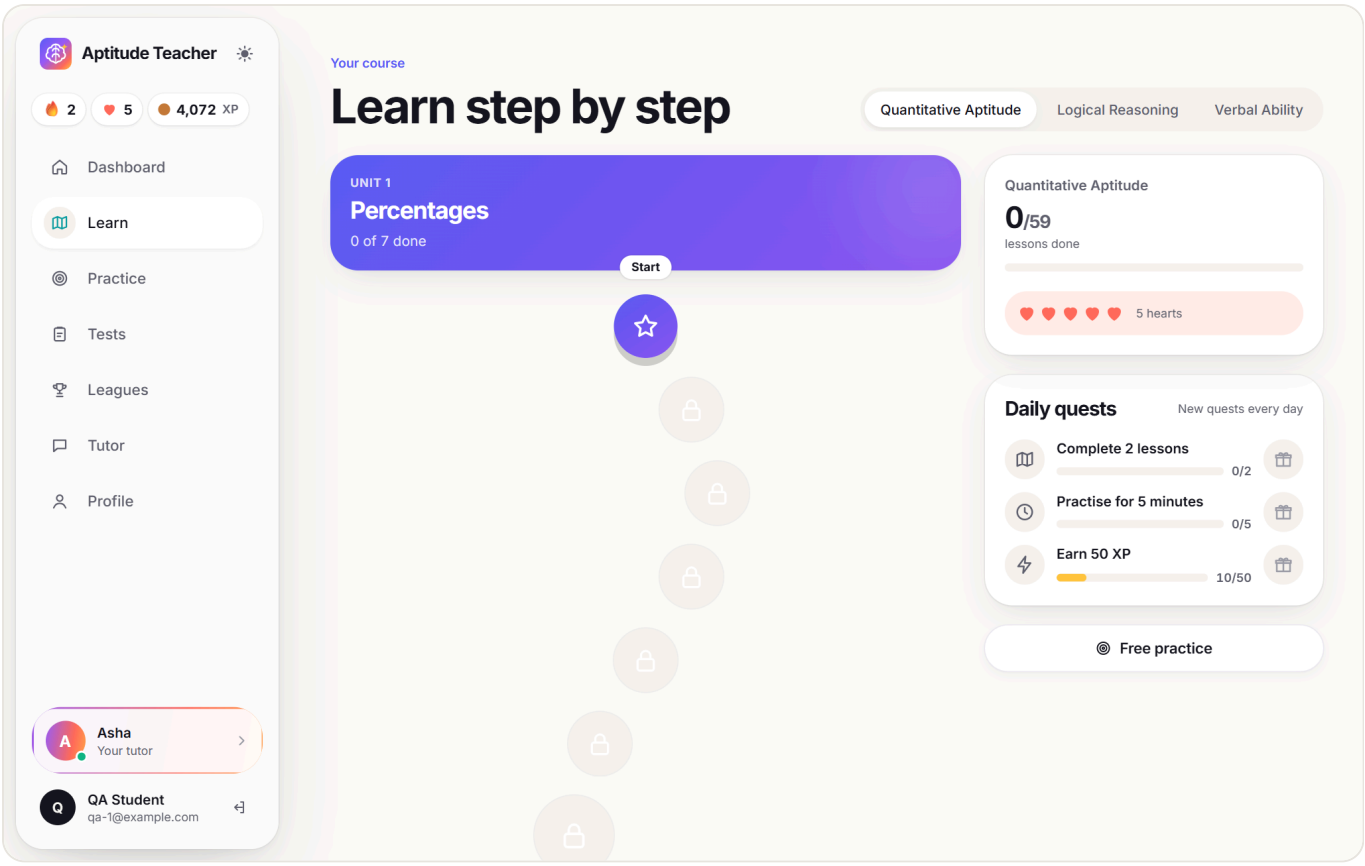
CHAPTER 2

A tour of the app

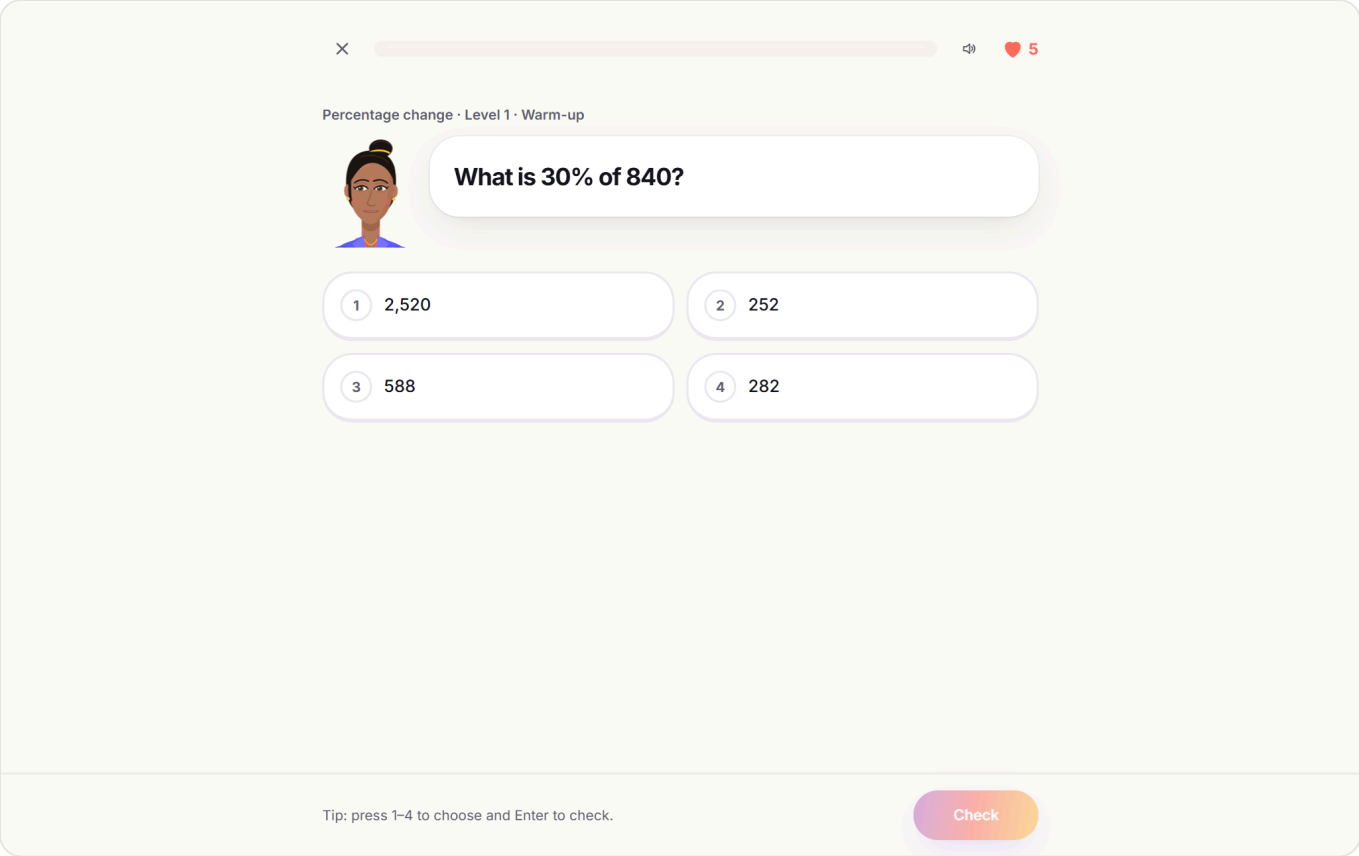
These are real screenshots of the app running on a computer with a test account.



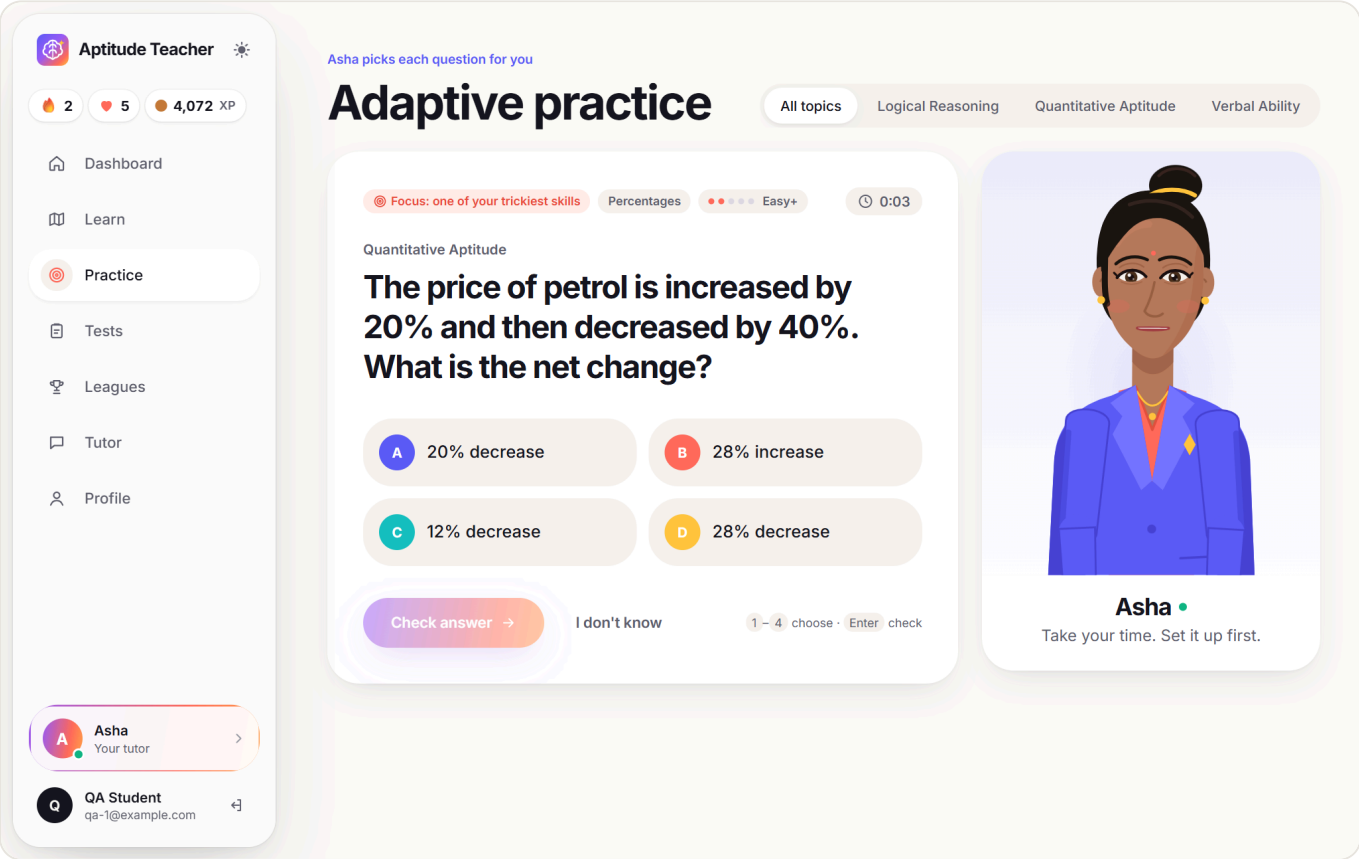
Dashboard. Greeting, a tip from the tutor about the student's weakest skill, today's goal rings (goal, accuracy, streak), level, streak, accuracy and average time per question. The chips at top-left show streak 🔥, hearts ❤️ and XP.



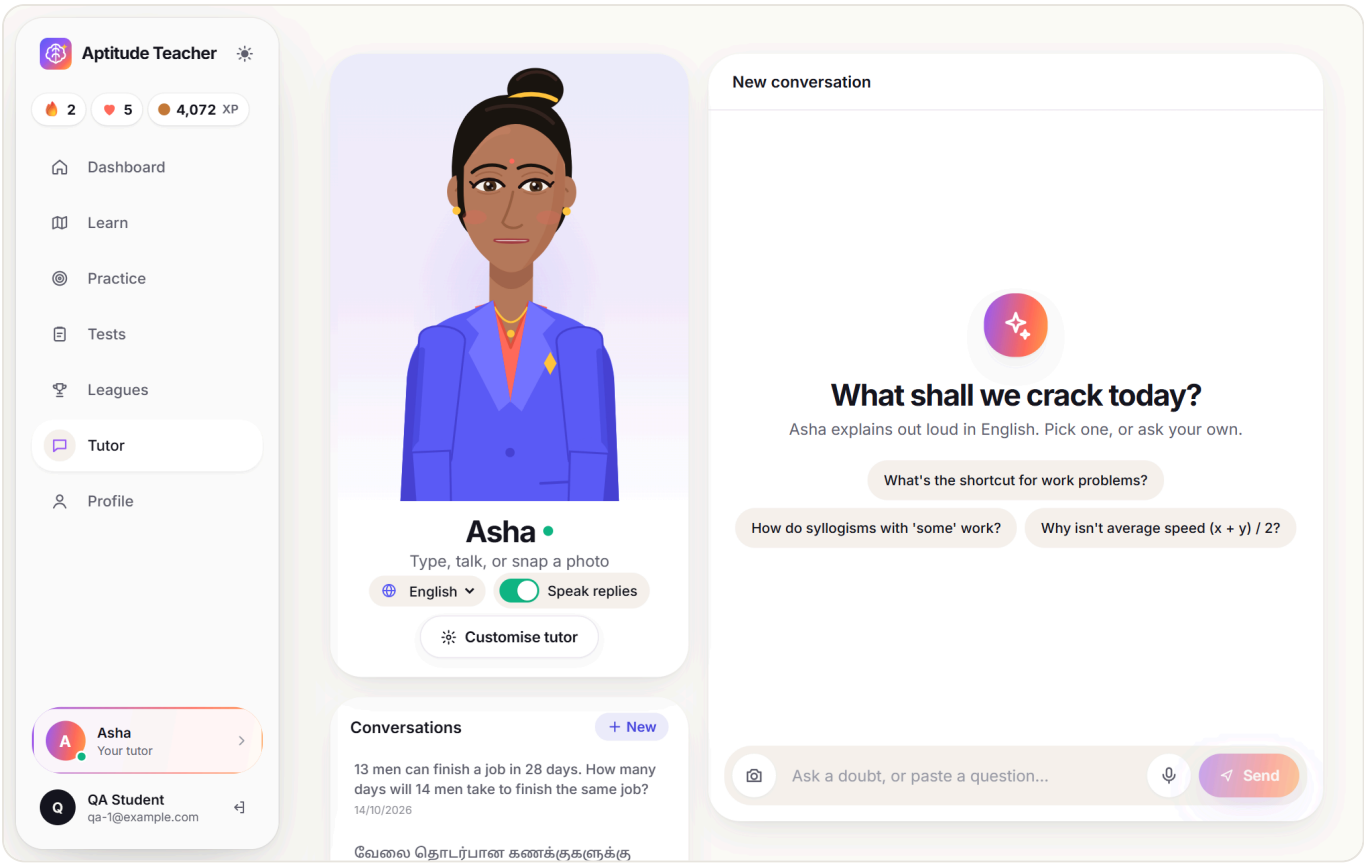
Learn (the path). Like Duolingo: each section is a winding path of lessons. Finish one to unlock the next. Right side: progress, hearts and today's three quests.



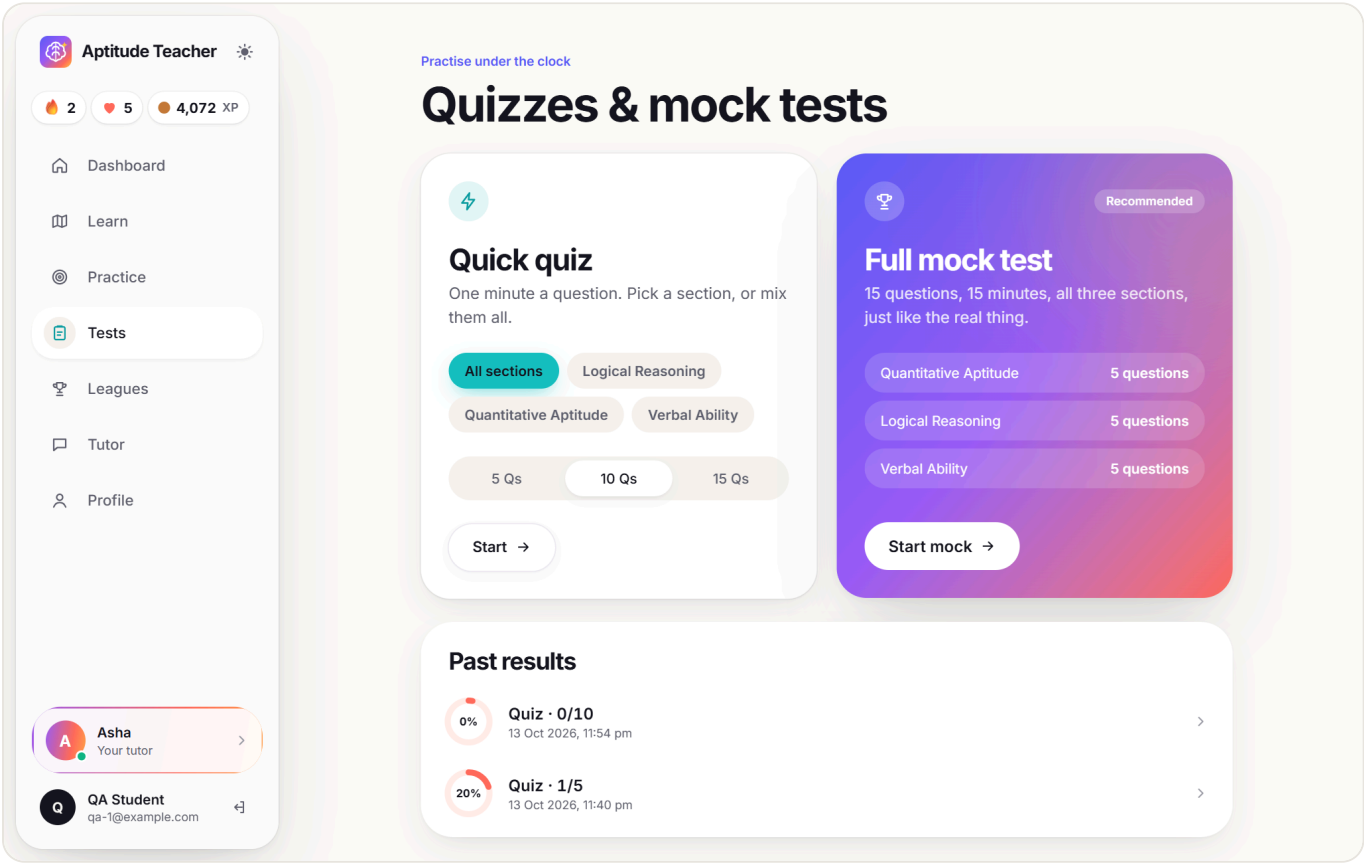
A lesson. 8 questions, one at a time. The tutor reads the question; the student picks an answer (keys 1–4 work too) and presses Check. A wrong answer costs a heart and the question comes back later in the lesson.



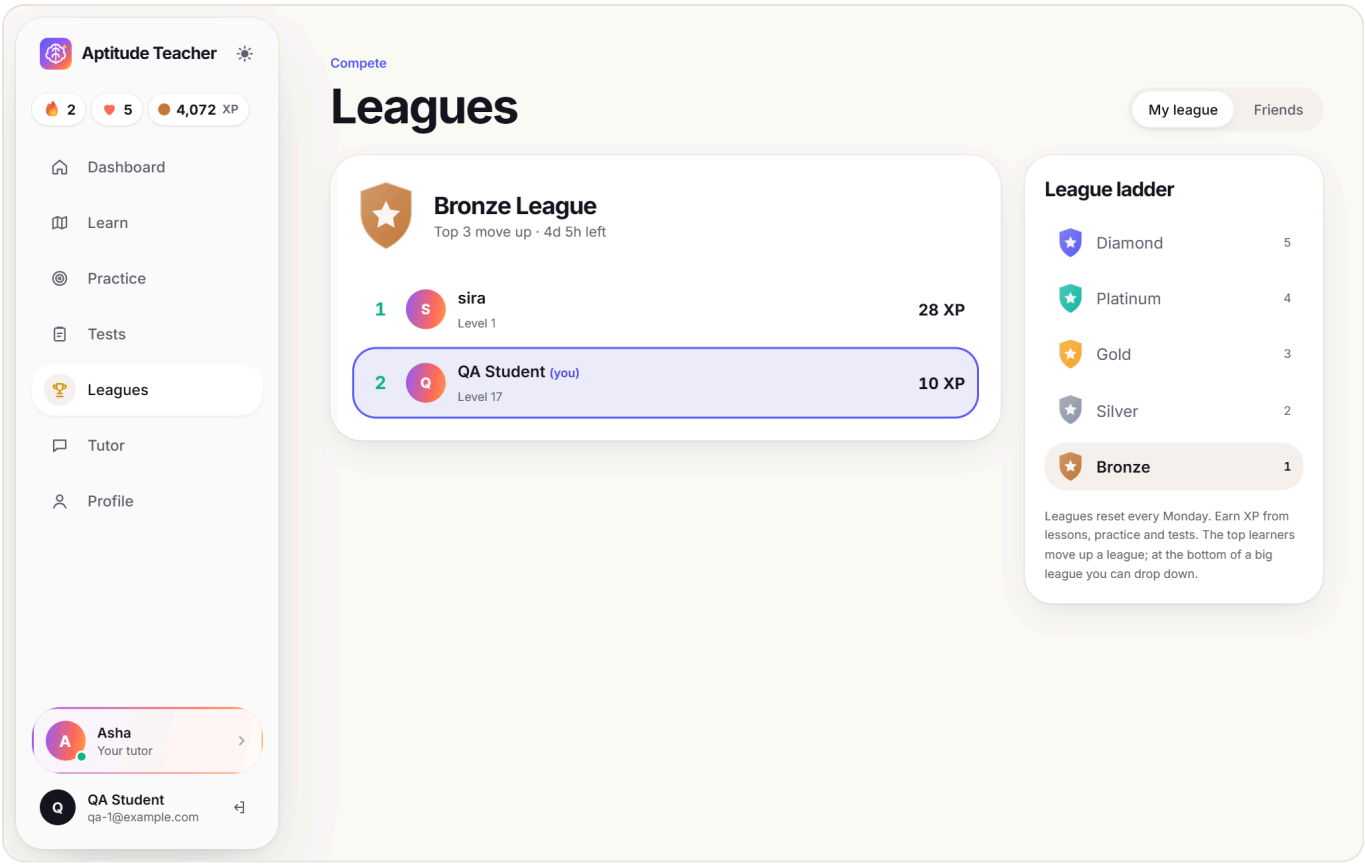
Practice. Free, adaptive practice. The app picks the next question from the student's weakest skill and brings back re-tests. Correct answers here earn hearts back.



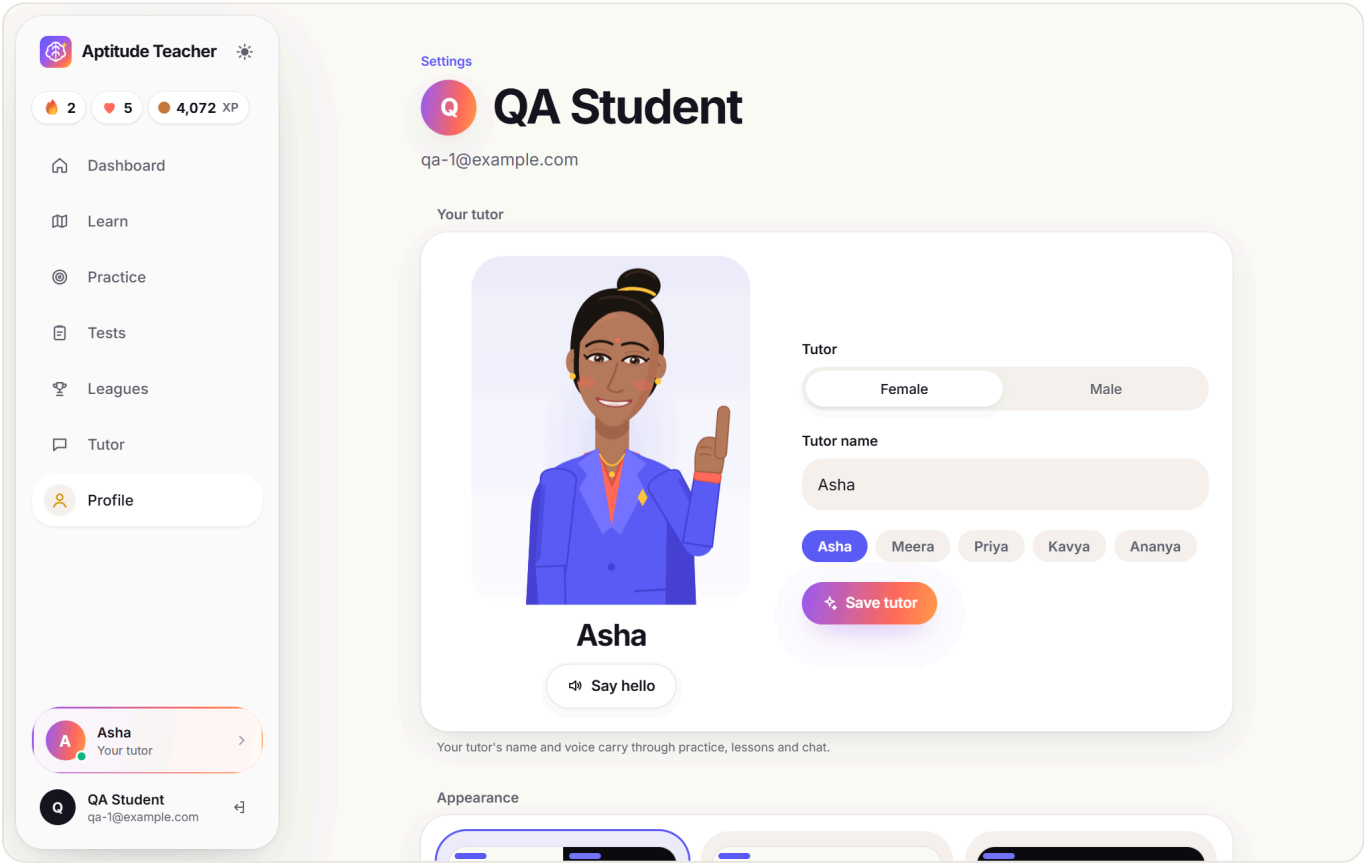
Tutor. Chat with Asha or Arjun by typing, speaking (microphone) or taking a photo of a question. Replies are spoken aloud sentence by sentence with lip-sync and gestures.



Tests. Timed quizzes on one topic, and 15-question mock tests across sections, each with a report afterwards.

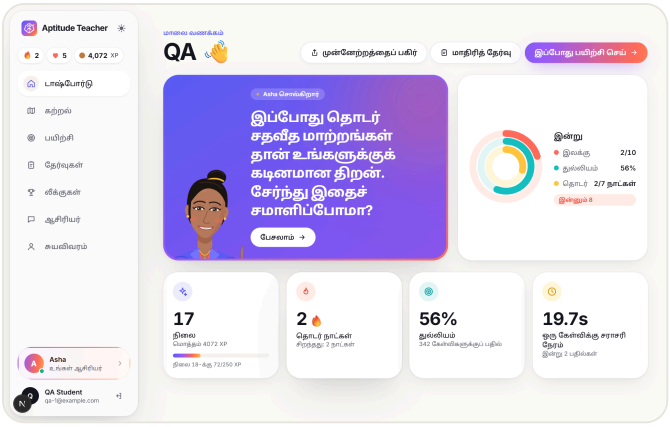


Leagues. Weekly leaderboard (Bronze → Silver → Gold → Platinum → Diamond) and a Friends tab where students add each other with a 6-character friend code.

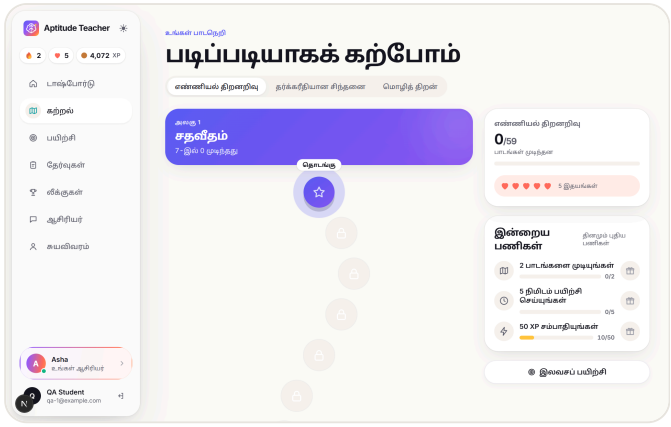


Profile. Display name, app language (English, Hindi, Tamil, Telugu) and the tutor (name and Asha/Arjun look).

Four languages, light and dark, phones too



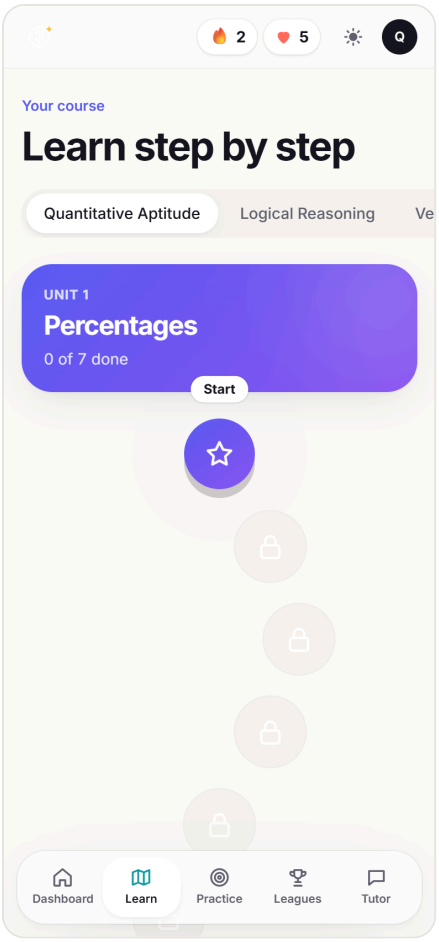
The same dashboard in Tamil. The tutor's tips, menus and explanations follow the student's language.



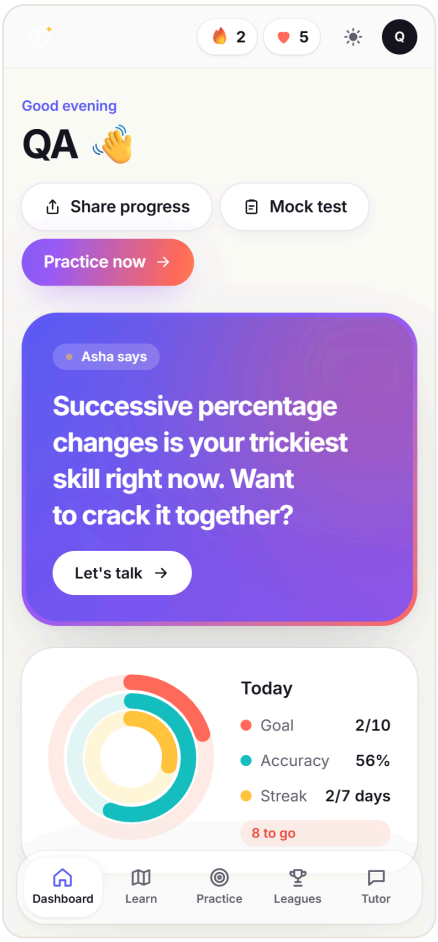
The learning path in Tamil.



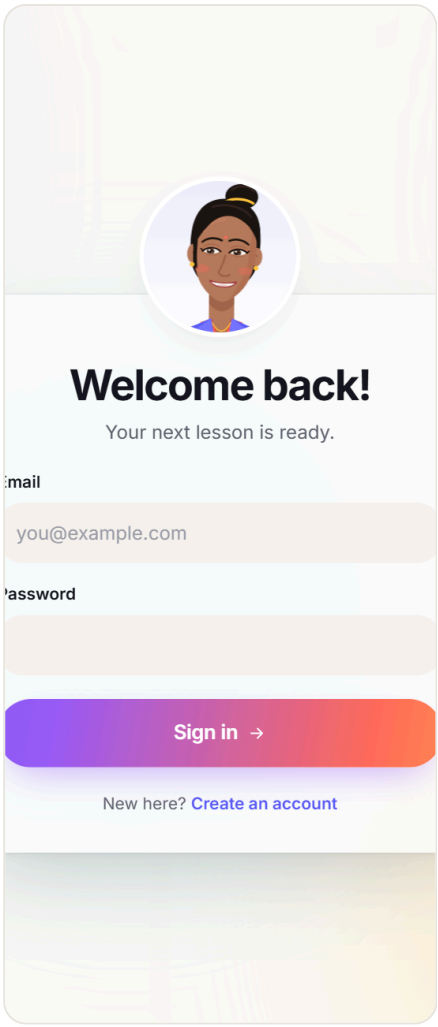
Dark mode (follows the device setting, or switch with the sun/moon button).



On a phone: the path, with a tab bar at the bottom.



On a phone: the dashboard.

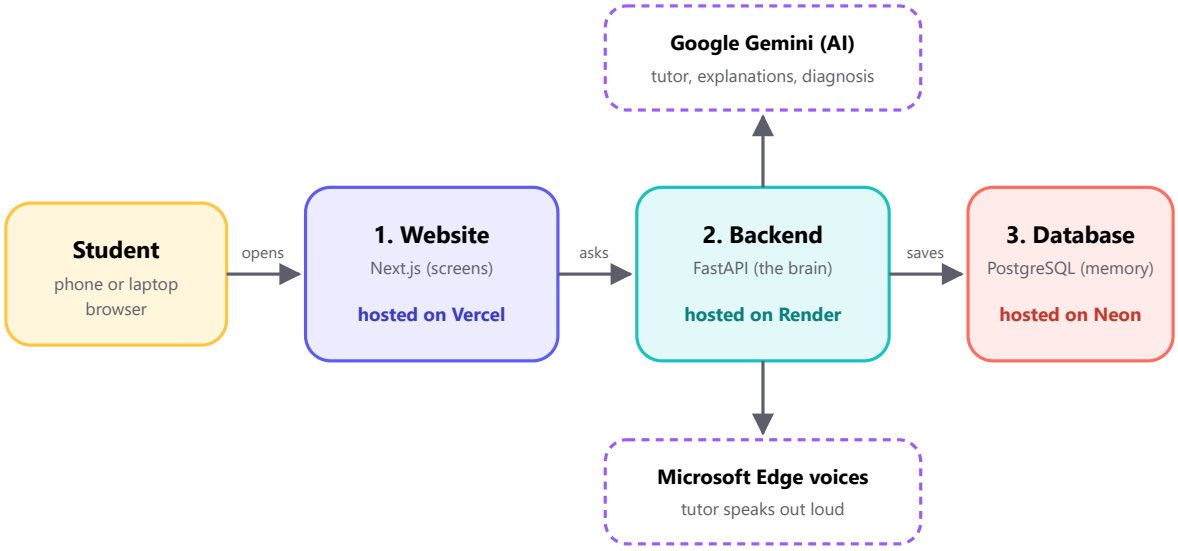


Sign in / create account. No email verification is needed.

CHAPTER 3

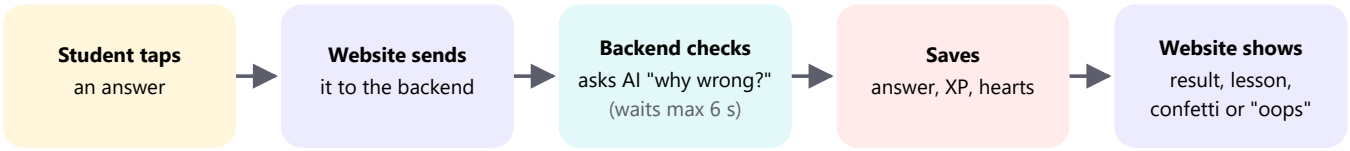
The big picture

The app is made of three parts that live in three different free services, plus two outside helpers.



Part	What it does, in plain words
1. Website (frontend)	Everything the student sees and taps: pages, buttons, animations, the tutor character, sounds. It does not store anything important itself; it asks the backend.
2. Backend (API)	The brain. It handles sign-up and login, picks the next question, checks answers, gives XP and hearts, runs leagues, talks to the AI and makes the tutor's voice. It keeps the AI key secret.
3. Database	The memory. Accounts, every answer, progress, XP, streaks, badges, friends and tutor chats are stored here, so nothing is lost when a student changes phone.
Gemini (optional)	Google's AI. Writes tutor replies, step-by-step explanations in four languages, short lessons and new questions; reads photos of questions. Without a key the app still works with built-in answers.
Edge voices	Free natural voices used to make the tutor speak. No key needed.

What happens when a student answers a question



If the AI is slow or busy, the backend does not wait more than 6 seconds: it answers with its own built-in explanation, so the student is never stuck.

CHAPTER 4

Tech stack and why

Every choice was made for three reasons: it must be **free**, it must be **reliable for beginners**, and it must be **popular** so help is easy to find online.

Tool	What it is	Why we chose it
Website (folder `apps/web`)		
Next.js 16 + **React 19**	The most popular framework for building websites with React.	Fast pages, works on phones, Vercel (made by the same company) hosts it for free with zero setup.
TypeScript	JavaScript with "types" that catch mistakes early.	Fewer bugs; the editor warns before something breaks.
Tailwind CSS 4	A way to style pages with short class names.	Quick to build a consistent, polished design, light and dark mode.
Motion, **Lenis**	Animation and smooth-scroll libraries.	The playful, high-end feel: page transitions, confetti, springy buttons.
Own SVG tutor rig	The animated Asha/Arjun characters, drawn in code.	Loads instantly, works on cheap phones, no downloads; lip-sync, 15 expressions, 20 gestures.
PWA	"Install" the website like an app.	Phone app feel without the Play Store.
Backend (folder `apps/api`)		
Python 3.13 + **FastAPI**	A modern, fast way to build an API in Python.	Simple to read, very popular, automatic API documentation page at `/docs`.
SQLAlchemy	Lets Python talk to the database.	The same code works with SQLite (on a laptop) and PostgreSQL (online).
SQLite (local)	A database that is just one file.	Zero setup for running on a laptop: the file `aptitude.db` is created automatically.
PostgreSQL (online)	A professional database server.	Online hosts wipe files on restart, so live data must live in a real database. Neon gives one free.
Own login (JWT)	Passwords are scrambled (PBKDF2) and the app hands out signed login tokens.	No paid login service and no extra account needed.
Google Gemini	Google's AI models, via the `google-genai` library.	Has a genuinely free tier; one key covers chat, explanations, translation, question writing and photo reading.
edge-tts	Free Microsoft Edge neural voices.	Natural Indian voices in all four languages, no key. Azure or Google voices can be added later.

pytest	Automatic tests.	250 tests check the app in seconds after every change, without calling the real AI.
Hosting (all free plans)		
GitHub	Where the code is stored.	Render and Vercel read the code from here and update the live app automatically.
Vercel	Hosts the website.	Best home for Next.js, free, fast in India, automatic HTTPS.
Render	Hosts the backend.	Free Python hosting, reads the ready-made setup file `render.yaml`.
Neon	Hosts the PostgreSQL database.	Free forever plan (not a 30-day trial), Singapore region close to India.

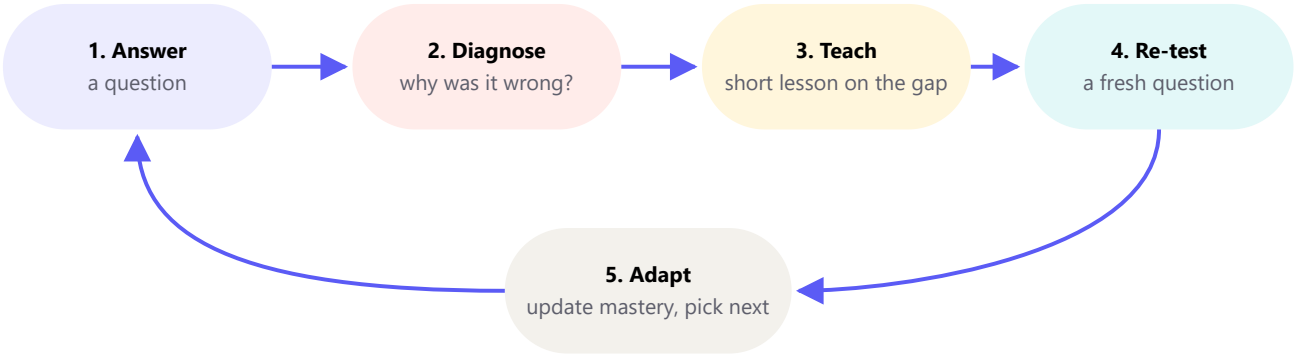
Why not put everything on Vercel?

Vercel is made for websites. The backend needs to run continuously, keep a database connection and hide the AI key, which is what Render and Neon are for. If the AI key were inside the website, anyone could copy it.

CHAPTER 5

How the main parts work

5.1 The learning loop (the heart of the product)



Mistake type	How it is spotted	What the app does
Careless slip	Right method, small arithmetic or reading slip (the wrong option is a "near miss").	Gentle reminder, no lesson.
Concept gap	Wrong method or misunderstood idea.	Short lesson on that idea + re-test.
Partial method	Right start, stopped early or slipped one step.	Lesson on the missing step + re-test.
Guess	An unlikely option picked very fast.	Encourages slowing down.

With a Gemini key the AI decides the type by reading the wrong option; without a key (or if the AI is slow) the app estimates it from the time taken. Each skill has a **mastery score**; the next practice question comes from pending re-tests first, then the weakest skill.

5.2 The course (Learn tab)



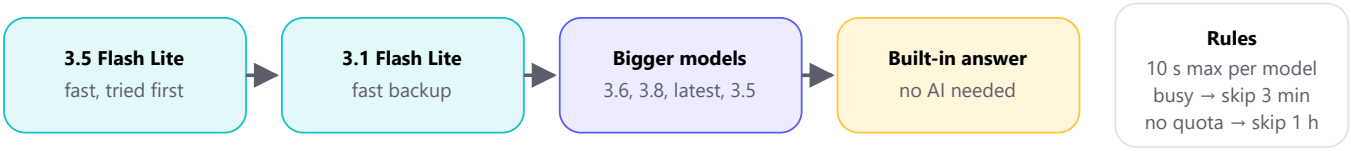
- Lessons unlock one after another. Stars: 3 stars for 90%+ right on the first try, 2 for 70%+, otherwise 1.
- Questions in a lesson are ones the student has not seen, near the right difficulty. If a skill runs out, the question templates write brand-new ones.
- A wrong answer in a lesson returns at the end, so every lesson ends with all questions right.

5.3 The game layer

Feature	Rules
❤️ Hearts	Maximum 5. A wrong answer in a lesson costs 1. One heart refills every 4 hours, and a correct answer in free Practice earns one back.
⚡ XP and levels	XP for every correct answer (more for harder questions), lesson bonus 10 + 5 per star, +20 for a checkpoint. A new level every 250 XP.
🔥 Streak	Days in a row with practice. A daily reminder can nudge the student.
🎯 Daily quests	Three quests per day (earn XP, finish lessons, perfect lesson, answers in a row, minutes practised); each claimed quest gives 15 XP.
🏆 Leagues	Weekly XP leaderboard: Bronze, Silver, Gold, Platinum, Diamond. Top 3 move up each week; bottom 3 move down (only in leagues of 10+).
👥 Friends	Every account has a 6-character friend code to follow each other.
🏅 Badges	12 badges (first lesson, perfect lesson, unit cleared, league promotion, streaks…).

5.4 The AI and how it stays fast

The free Gemini plan has limits (about 20 requests a day for each bigger model) and its bigger models are sometimes overloaded. So the backend keeps a **list of models** and moves down the list when one is busy, slow or out of quota.



- **Every AI feature has a fallback.** No key, no internet, or all models busy: the app still teaches with its own explanations and a time-based diagnosis.
- The tutor reply is **streamed**: the first sentence appears (and is spoken) while the rest is still being written.
- The tutor has a personality (playful, kind about mistakes) and tags each sentence with an emotion so the character acts it out.

5.5 Four languages

- **Menus and buttons:** full word lists for each language in `apps/web/src/lib/locales/` (`en.ts, hi.ts, ta.ts, te.ts`).
- **Questions:** ready-made translations in `apps/api/app/translations/` (hi, ta, te). Questions without a ready translation are translated live by the AI.
- **Tutor:** replies in the student's chosen language, in native script, with normal digits.

5.6 Voice

- **Tutor speaking:** the backend turns each sentence into audio with the free Edge voices (male/female voices for each language), with word timings for lip-sync. Each spoken sentence is saved, so it keeps working if the voice service is down.
- **Student speaking:** the microphone button uses the browser's own speech recognition (works best in Chrome and Edge).

5.7 Accounts and safety

- Passwords are never stored as typed; only a scrambled version (PBKDF2-SHA256).
- After login the browser keeps a signed token (valid 7 days). The signing secret is `JWT_SECRET`; online it is generated automatically by Render.
- The backend only accepts requests from the website addresses listed in `CORS_ORIGINS`.

CHAPTER 6

Project map

One repository ("monorepo") holds both the website and the backend.

```
aptitudeteacher/
├── start.bat                ← double-click to run everything on Windows
├── render.yaml              ← ready-made setup for Render (backend hosting)
├── README.md                ← short developer readme
├── HANDOFF.md               ← detailed history of decisions (for developers)
└── apps/
    ├── api/                 ← BACKEND (Python / FastAPI)
    │   ├── requirements.txt ← list of Python packages
    │   ├── .env.example     ← template for the private settings file .env
    │   ├── app/
    │   │   ├── main.py      ← starts the API, connects all routes
    │   │   ├── config.py    ← reads settings (keys, database address…)
    │   │   ├── auth.py      ← passwords and login tokens
    │   │   ├── ai.py        ← every Gemini call + fallbacks + model list
    │   │   ├── tts.py       ← tutor voice
    │   │   ├── offline_tutor.py ← tutor replies when there is no AI
    │   │   ├── seed.py      ← creates tables and loads questions on start
    │   │   ├── db/          ← database tables (models.py) and connection
    │   │   ├── engine/      ← mastery.py, course.py, game.py, gamification.py
    │   │   ├── routes/      ← the API addresses: auth, learn, practice, quizzes,
    │   │   │                   tutor, progress, social (leagues/friends), speech
    │   │   ├── question_bank/      ← hand-written questions (JSON files)
    │   │   ├── question_templates/ ← ~110 generators of fresh questions
    │   │   └── translations/       ← question translations hi / ta / te
    │   └── tests/           ← 250 automatic tests
    └── web/                 ← WEBSITE (Next.js)
        ├── package.json     ← list of JavaScript packages
        ├── public/          ← icons, offline helper (sw.js)
        └── src/
            ├── app/         ← one folder per page: dashboard, learn, lesson,
            │                   practice, tests, leagues, tutor, profile, login…
            ├── components/  ← shared pieces: AppShell (menus), TutorFigure…
            └── lib/         ← api.ts (talks to backend), locales/ (4 languages),
                                game.ts, tutorRig.ts, voice, theme…
```

Files that are private and never in GitHub

`apps/api/.env` (keys and passwords) and `apps/api/aptitude.db` (the local database). They are listed in `.gitignore` so Git ignores them. Never remove them from that list.

CHAPTER 7

Install on a computer (from scratch)

Do this to run the app on your own computer, to try it or to change it. Time: about 20 minutes, most of it downloads.

Step A — Install three free programs (once)

Program	Where	Important while installing
Git	git-scm.com/downloads	Keep all default options.
Node.js 20 or newer (LTS)	nodejs.org	Default options.
Python 3.11 or newer (3.13 recommended)	python.org/downloads	**Windows: tick "Add python.exe to PATH"** on the first screen.

Check: open **Command Prompt** (Windows) or **Terminal** (Mac) and type these one by one. Each should print a version number.

```
git --version
node --version
python --version      # on Mac: python3 --version
```

Step B — Get the code

The repository is private, so the owner must first invite you: on GitHub, repository → **Settings** → **Collaborators** → **Add people**. Accept the email invite, then:

```
git clone https://github.com/rhythmoflens007-byte/aptitudeteacher.git
cd aptitudeteacher
```

Step C — (Optional) Turn on the AI

1. Go to **aistudio.google.com/apikey**, sign in with a Google account, click **Create API key**, copy it.
2. In the folder `apps/api`, make a copy of `.env.example` and name the copy `.env`.
3. Open `.env` in Notepad and paste the key after `GEMINI_API_KEY=` (no spaces, no quotes). Also replace `dev-only-change-me` after `JWT_SECRET=` with any long random text. Save.

> **No key? No problem.**
>
> Without `.env` the app runs fully on built-in answers. The AI only makes the tutor and explanations smarter.

Step D — Start the app

Windows (easiest)

Double-click `start.bat` in the main folder. The first time it installs everything (a few minutes). It then opens two black windows (backend and website, leave them open) and the browser at **http://localhost:3000**. To stop the app, close the two black windows.

Mac / Linux, or by hand

First time only:

```
cd apps/api
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt     # Windows: .venv\Scripts\python
cd ../web
npm install
```

Every time, in **two** terminal windows:

```
# window 1: backend
cd apps/api
.venv/bin/python -m uvicorn app.main:app --port 8000

# window 2: website
cd apps/web
npm run dev
```

Open **http://localhost:3000** and create an account. The local database file is created and filled with the 1,212 questions automatically on the first start.

Step E — Check it works

- Open **http://localhost:8000/health**. You should see `"status":"ok"` and `"ai_enabled":true` (or `false` without a key).
- Open **http://localhost:8000/docs** to see every backend address (useful for developers).
- Run the automatic tests (should say **250 passed**):

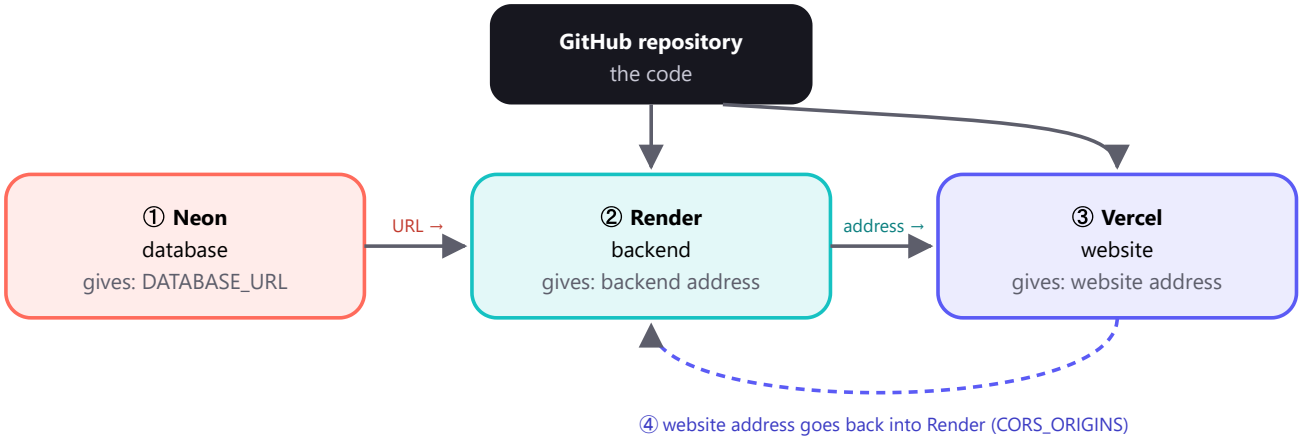
```
cd apps/api
.venv/bin/python -m pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q
```

> **Reset local data**
>
> Stop the app and delete `apps/api/aptitude.db`. It is recreated on the next start. This deletes every local account.

CHAPTER 8

Deploy online from scratch

Put the app on the internet so anyone can use it. Everything below is free. Time: about 30 minutes. You need a GitHub account with access to the repository.



Before you start: keep secrets secret

You will handle three secrets: the **Neon connection string**, the **Gemini key**, and (automatically) the **login secret**. Paste them only into the Neon/Render/Vercel websites. Never into chat, email, screenshots or code files.

① Database on Neon (5 minutes)

1. Go to **neon.tech** → **Sign up** → **Continue with GitHub**.
2. Create a project: name `aptitude-teacher`, Postgres version: the default, region **AWS Asia Pacific (Singapore)**. Click **Create**.
3. On the project dashboard click **Connect**. Copy the **connection string**. It looks like `postgresql://user:password@ep-xxxx.ap-southeast-1.aws.neon.tech/neondb?sslmode=require`
4. Keep it in a safe place for the next step (for example a password manager). It contains the database password.

② Backend on Render (10 minutes)

1. Go to **render.com** → **Get Started** → **GitHub**. Allow Render to see the `aptitudeteacher` repository (choose "Only select repositories" and pick it).
2. In the Render dashboard click **New +** → **Blueprint**. Pick the `aptitudeteacher` repository, branch `main`. Render finds the file `render.yaml` and shows one service: **aptitude-teacher-api** (free plan, Singapore).
3. Render asks for three values:

Name	What to paste
`DATABASE_URL`	The Neon connection string from step ①.
`GEMINI_API_KEY`	The Gemini key (aistudio.google.com/apikey). Can be left empty; the app works without AI.
`CORS_ORIGINS`	Type `http://localhost:3000` for now. It is replaced in step ④.

   `JWT_SECRET`, `ENVIRONMENT` and `PYTHON_VERSION` are filled automatically.

4. Click **Apply** / **Deploy Blueprint**. Watch the **Logs**: it installs packages, then shows `Application startup complete`. The first start takes 3–6 minutes because it loads all questions into the new database.
5. At the top of the service page, copy the address, e.g. `https://aptitude-teacher-api.onrender.com`.
6. Test it: open that address + `/health` in the browser. You should see `{"status":"ok","environment":"production","ai_enabled":true}`.

> **The free backend sleeps**
>
> After 15 minutes with no visitors Render puts the free backend to sleep. The next visitor waits about 1 minute while it wakes up (the website may show "took too long" once; refreshing fixes it). Paid plans (about $7/month) never sleep. See chapter 10 for a free workaround.

③ Website on Vercel (5 minutes)

1. Go to **vercel.com** → **Sign Up** → **Continue with GitHub**. Choose the free **Hobby** plan.
2. Click **Add New...** → **Project** → **Import** next to `aptitudeteacher` (if missing, click "Adjust GitHub App Permissions" and allow it).
3. **Root Directory**: click **Edit** and choose `apps/web`. Framework is detected as **Next.js**; leave build settings as they are.
4. Open **Environment Variables** and add:
   `NEXT_PUBLIC_API_URL` = the Render address from step ② (no slash at the end).
5. Click **Deploy**. After 1–3 minutes you get a link such as `https://aptitudeteacher.vercel.app`. Copy it.

Important

If you change `NEXT_PUBLIC_API_URL` later, you must redeploy the website (Vercel → Deployments → ⋯ → Redeploy), because this value is built into the website.

④ Connect them (2 minutes)

1. In Render → **aptitude-teacher-api** → **Environment** → edit `CORS_ORIGINS` → put the Vercel address, e.g. `https://aptitudeteacher.vercel.app`. For more than one address, separate with commas.
2. Click **Save, rebuild and deploy**. Wait until it is "Live".

⑤ Final check: is it working?

	Check	Expected
☐	Open the Vercel address	The home page loads.
☐	Create an account	You land on the dashboard. (No error "can't reach the server".)
☐	Learn → start the first lesson, answer a few	Questions appear, hearts and XP change.
☐	Practice → answer one wrongly	An explanation (and maybe a short lesson) appears within a few seconds.
☐	Tutor → type "hi"	A reply in a few seconds, spoken aloud if "Speak replies" is on.
☐	Profile → switch to Tamil	Menus and tutor switch to Tamil.
☐	Open the site on a phone	Works; browser menu offers "Install app" / "Add to Home screen".
☐	Sign out and sign in again	Progress is still there (it is saved in Neon).

First online run: watch for database errors

All tests run on SQLite. The online database (PostgreSQL) is supported by the code but had not been run live when this guide was written. If Render's Logs show an error mentioning `psycopg`, `value too long` or `column`, copy the red lines and give them to a developer; it is usually a one-line fix.

Custom domain (optional)

To use a name like `aptitudeteacher.in`: buy the domain, then Vercel → Project → Settings → Domains → Add. Remember to add the new address to `CORS_ORIGINS` on Render too.

CHAPTER 9

All settings in one table

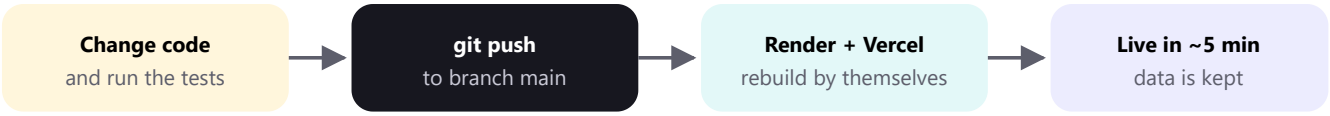
Settings ("environment variables") are values the app reads when it starts. Locally they live in `apps/api/.env`; online they are typed into Render or Vercel.

Name	Where	What it is	Type
`DATABASE_URL`	Render	Neon connection string. Locally empty = use the file `aptitude.db`.	secret
`GEMINI_API_KEY`	Render	Google AI key. Empty = AI off, built-in answers used.	secret
`JWT_SECRET`	Render	Signs login tokens. Render generates it. The backend refuses to start in production with the default value.	automatic
`ENVIRONMENT`	Render	`production` online, `development` locally.	automatic
`CORS_ORIGINS`	Render	Website address(es) allowed to use the backend, comma-separated.	public
`PYTHON_VERSION`	Render	3.13.14 (from `render.yaml`).	automatic
`GEMINI_MODEL`	optional	First AI model to try. Default `gemini-3.5-flash-lite`.	public
`GEMINI_FALLBACK_MODELS`	optional	Backup models, in order, comma-separated.	public
`TTS_PROVIDERS`	optional	Voice services in order. Default `edge` (free, no key). Options: `azure`, `google`.	public
`AZURE_SPEECH_KEY` / `_REGION`	optional	Only if using Azure voices.	secret
`GOOGLE_TTS_API_KEY`	optional	Only if using Google voices.	secret
`NEXT_PUBLIC_API_URL`	Vercel	The backend address. Locally defaults to `http://localhost:8000`.	public

CHAPTER 10

After going live

Updating the app




```
git add -A
git commit -m "Describe what changed and why"
git push origin main
```

New tables or columns are added to the database automatically on start; existing data is kept. New questions added to `apps/api/app/question_bank/*.json` are loaded on the next start.

Free plan limits to know

Service	Free limit	What it means / what to do
Render	Sleeps after 15 min idle; 750 hours/month.	First visitor waits ~1 minute. Optional: a free pinger (e.g. cron-job.org) calling `/health` every 10 minutes keeps it awake; one always-on service fits in 750 hours. Or upgrade (~$7/month).
Neon	About 0.5 GB storage; pauses when idle and wakes in about a second.	Enough for thousands of students. Check usage on the Neon dashboard.
Vercel	Hobby plan: generous traffic, for non-commercial use.	If the app starts earning money, move to Vercel Pro.
Gemini	~20 requests per day for each bigger model; lite models allow more.	The app rotates models automatically. With many students, add billing in Google AI Studio (pay per use, low cost).

Backups

- Neon keeps a short history and can restore the database to an earlier moment (Neon dashboard → **Backup & Restore**). Check how far back the free plan goes.
- For a full copy, use Neon's dashboard or the standard `pg_dump` tool with the connection string, and store the file somewhere private. Do this weekly once real students use the app.

If a key leaks (changing keys)

- **Gemini key:** aistudio.google.com/apikey → delete the old key → create a new one → paste it in Render → Environment → `GEMINI_API_KEY` → Save.

- **Database password:** Neon → Roles → reset password → copy the new connection string → update `DATABASE_URL` in Render.
- **Login secret:** Render → Environment → `JWT_SECRET` → Generate a new value → Save. Everyone is signed out once.

Where to look when something is wrong

- **Render → service → Logs**: backend errors (in red).
- **Vercel → project → Deployments / Logs**: website build errors.
- **Backend address + /health**: is the backend alive, is AI on.

CHAPTER 11

Troubleshooting

I see...	Do this
"Can't reach the server" / "That took too long" on the live site	The free backend was asleep. Wait one minute and refresh. If it continues: open the backend `/health`; if that fails, check Render Logs.
Sign-up fails with a network or CORS error (browser console says "blocked by CORS policy")	`CORS_ORIGINS` on Render does not exactly match the website address. Use `https://`, no slash at the end, then Save and redeploy.
The website still calls `localhost:8000`	`NEXT_PUBLIC_API_URL` missing in Vercel or added after the build. Add it and **Redeploy**.
Render log: "Set JWT_SECRET to a long random value"	`JWT_SECRET` is missing or the default. Render → Environment → add `JWT_SECRET` → Generate.
Render log mentions `password authentication failed` or `could not translate host name`	`DATABASE_URL` is wrong. Copy it again from Neon → Connect (the full line).
`/health` shows `"ai_enabled":false`	`GEMINI_API_KEY` is empty on Render. Add it and save.
Tutor says it is busy / AI unavailable	Google's free models are overloaded or the daily quota is used. The app tries other models and falls back automatically; wait a few minutes. Quotas reset daily.
No voice from the tutor	Check "Speak replies" is on and the device is not muted. Voices come from the backend; check Render Logs for `tts` errors. Voice input (mic) works best in Chrome/Edge.
Vercel build fails	Root Directory must be `apps/web`. Read the red lines in the build log. Locally, `npm run build` in `apps/web` shows the same error.
Locally: `python` or `npm` "not recognized"	Reinstall Python with "Add python.exe to PATH" ticked; reinstall Node.js; open a new Command Prompt.
Locally: "port 8000/3000 already in use"	The app is already running in another window. Close the old black windows, or restart the computer.
Locally: changes to Python code not showing	Stop and start the backend window again.

CHAPTER 12

Security rules

Never

- Put keys or passwords in code, GitHub, chat, email or screenshots.
- Commit `.env` or any `.db` file.
- Put the Gemini key in Vercel or anywhere in the website.
- Run production with `JWT_SECRET=dev-only-change-me` (the app blocks this).
- Make the GitHub repository public without checking its history for secrets.

Always

- Paste secrets only into Neon / Render / Vercel settings pages.
- Use "Continue with GitHub" and turn on two-factor login on GitHub, Render, Vercel, Neon and Google.
- Change a key immediately if it may have leaked (chapter 10).
- Keep `CORS_ORIGINS` limited to your own website addresses.
- Run the tests before pushing changes.

Not built yet (recommended before a big public launch): privacy policy and terms pages, password reset by email, error-alert monitoring, scheduled database backups.

CHAPTER 13

Glossary and cheat sheet

Word	Meaning
Frontend / website	The part people see in the browser (here: Next.js on Vercel).
Backend / API	The program behind the website that does the work and stores data (here: FastAPI on Render).
Database	Where data is kept permanently (SQLite file locally, PostgreSQL on Neon online).
Deploy	Putting a version of the app online.
Repository (repo)	The project folder stored on GitHub, with its full history.
Commit / push	Save a change in Git / send saved changes to GitHub.
Environment variable	A named setting given to the app when it starts (like `GEMINI_API_KEY`).
`.env` file	The private local file holding environment variables. Never shared.
API key	A secret password that lets the app use a service (Gemini).
CORS	A browser safety rule: the backend only answers websites on its allowed list.
JWT / token	A signed "pass" the browser shows the backend to prove who is logged in.
Localhost	"This computer". `localhost:3000` is the website running on your own PC.
PWA	A website that can be installed on a phone like an app.
Free tier	The no-cost plan of a service, with limits.
Cold start	The wait while a sleeping free backend wakes up.

Cheat sheet

I want to…	Do
Run locally (Windows)	Double-click `start.bat` → http://localhost:3000
Run backend tests	`cd apps/api` → `.venv/Scripts/python -m pytest -q`
Check the website builds	`cd apps/web` → `npm run build`
See backend addresses	http://localhost:8000/docs
Update the live app	`git push origin main` (auto-deploys)
Is the live backend OK?	Open `https://<render-address>/health`

Change a live setting	Render → Environment (backend) / Vercel → Settings → Environment Variables (website, then Redeploy)
Add questions	Edit `apps/api/app/question_bank/*.json`, push; loaded on next start

Summary in one breath

Aptitude Teacher is a Duolingo-style, four-language aptitude tutor. The website (Next.js) runs on Vercel, the brain (FastAPI) on Render, the memory (PostgreSQL) on Neon, the AI is Google Gemini, all on free plans. To deploy: create the Neon database, deploy the Render blueprint with the database URL and Gemini key, deploy `apps/web` on Vercel with the backend address, then put the website address back into Render.